Loop — AI Mock Interview Agent

Architecture Document

Version 1.0 · May 2026

Abstract. Loop is a multi-agent AI platform that parses a candidate's résumé, infers realistic target roles with cited evidence, and conducts an adaptive voice-driven mock interview while continuously analysing the candidate's eye contact, posture, engagement, and stress signals from the webcam feed. After the interview, the system synthesises a five-dimension feedback report including STAR-method analysis of behavioural answers and a specific coaching plan, then recommends live job postings ranked by semantic match to the candidate's résumé. The platform was designed and built solo for the IPHIPI Hackathon, with explainability and on-device inference as primary design goals.

1. Problem Summary

Existing mock-interview tools fall into two categories, both inadequate. Static-list tools cycle through hardcoded questions with no adaptation to the candidate; large-language-model demos generate questions but provide no visible reasoning, no feedback loop, and no signal that the system actually learned anything about who it was interviewing.

Loop's design goal is to make the agent's reasoning *visible* at every step. When the system infers a target role, it cites the exact résumé bullets that justified the inference. When the orchestrator adapts question difficulty, it surfaces a one-line explanation ("Scored 0.42 — dropping difficulty to 2"). When the feedback synthesiser flags a weakness, it quotes the literal phrase from the candidate's answer that triggered the flag. The candidate finishes a session knowing not only their score but precisely why they earned it and what to practice next.

1.1. User Journey

A complete session lasts eight to twelve minutes and follows seven stages:

1. **Upload résumé.** The candidate uploads a PDF.
2. **Role inference.** The system parses the résumé, extracts skills and experience, and proposes three target roles ranked by fit, each with cited evidence.
3. **Pick interviewer.** The candidate selects a personality (warm coach, FAANG-style technical drill, or behavioural specialist).
4. **Live interview.** The orchestrator generates questions dynamically, the candidate answers by voice, and the system scores each answer and adapts difficulty.
5. **Multimodal observation.** Throughout the interview, the browser runs MediaPipe face and pose detection to track eye contact, posture, engagement, and stress.
6. **Feedback synthesis.** On session end, the system produces a structured report combining the technical scoring, the multimodal observations, and STAR analysis of behavioural answers.
7. **Job matching.** The candidate is shown a ranked list of live job postings drawn from a public API, each with a per-posting rationale for why it fits their profile.

Figure 1: End-to-end user journey, executed in a single session.

2. High-Level Architecture

The system is organised into four layers: the browser client, a FastAPI HTTP layer, a service layer that owns the core business logic, and external or on-device dependencies. Figure 2 shows the full picture.

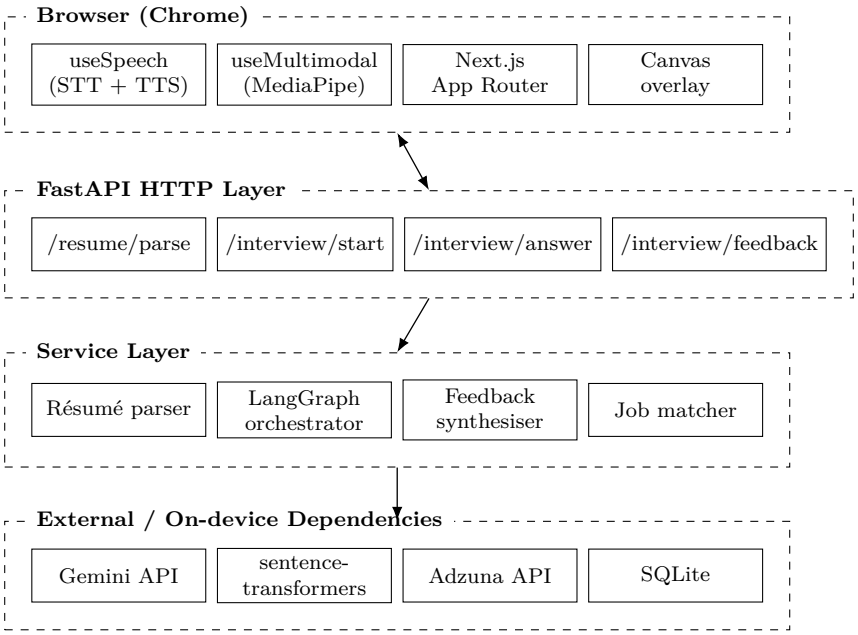

Figure 2: System architecture across four layers. Approximately 60 % of inference (speech, computer vision, embeddings) runs on the browser client. Only question generation, answer scoring, and feedback synthesis require calls to an external LLM.

2.1. On-device Inference

A deliberate design choice was to push as much inference as possible into the browser. Speech-to-text and text-to-speech use the Web Speech API, which is free and runs locally. Face and pose landmark detection use MediaPipe Tasks JavaScript, which downloads small WebAssembly models once and runs them on the CPU or GPU at approximately thirty frames per second. Semantic embeddings for job matching use the `sentence-transformers/all-MiniLM-L6-v2` model, an eighty-megabyte download that runs locally on the FastAPI host. Only the question generator, answer evaluator, and feedback synthesiser call out to a remote large language model. This approach keeps the cost per session near zero and avoids per-user API rate limits for the high-frequency operations.

3. The Agentic Loop

The core of the system is a LangGraph state machine that runs the interview as a cyclic sequence of three nodes plus one conditional edge. The state machine is shown in Figure 3.

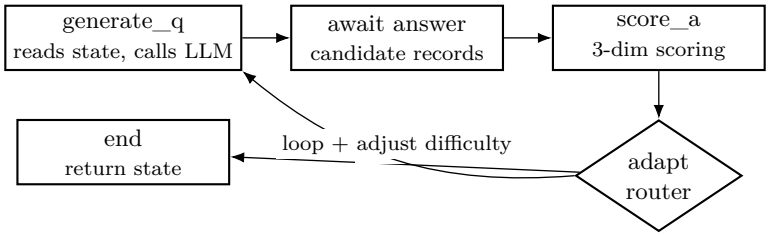

Figure 3: LangGraph state machine. The orchestrator node `adapt_router` decides whether to terminate the interview or loop back with adjusted difficulty. Its reasoning is written to the state as a one-line natural-language explanation and surfaced live to the candidate.

3.1. Routing Logic

The adapt router applies the following rules after each answer is scored:

- If the maximum number of questions has been reached, terminate.
- Otherwise, examine the score of the most recent answer:
  - If the overall score is below 0.4, reduce difficulty by one level (minimum 1) and set a flag indicating the next question should open with an encouraging phrase.
  - If the overall score is above 0.8, raise difficulty by one level (maximum 5).
  - Otherwise, hold difficulty constant.
- In all cases, write a one-line natural-language explanation of the decision to `state.last_decision`.

The router's decision string is rendered live in the interview UI so the candidate sees, in real time, why the next question changed in difficulty. This is the project's primary demonstration of agentic behaviour as distinct from a static question script.

3.2. Interviewer Personalities

The question generator's prompt is prepended with a personality directive loaded from a separate module. Three personalities are supported:

- **Mira** (warm coach). Patient, gives hints, restates questions when the candidate struggles. The default personality.
- **Marcus** (FAANG-style senior engineer). Cold, drilling, never validates effort. Presses on edge cases and scaling.
- **Priya** (behavioural specialist). Probes the how and why of decisions, explicitly requests STAR structure, asks for measurable outcomes.

Each personality also drives a distinct browser text-to-speech voice and a distinct SVG avatar. The encouragement-on-low-score behaviour described above is disabled for the Marcus personality so the candidate experiences the cold style consistently.

4. Scoring Approach

Scoring runs in two stages. The hot path scores each answer as it is submitted; the cold path synthesises the final report at session end. The final report aggregates signals from three independent sources: LLM semantic scoring of answer content, measured speech metrics from the browser, and computer-vision-derived multimodal signals.

4.1. Per-Answer Scoring (Hot Path)

After each candidate answer, the `answer_evaluator` agent returns three sub-scores in the range $[0, 1]$:

- **Correctness**: factual accuracy of technical claims.
- **Depth**: whether the answer probes the question at a sufficient level of detail.
- **Structure**: clarity of organisation, whether the answer follows a recognisable pattern such as STAR for behavioural questions.

These three sub-scores feed the adapt router and are stored on the session state for use by the synthesiser.

4.2. Speech and Visual Signals

In parallel with the interview, two independent measurement layers run on the browser:

- **Speech metrics.** Filler-word frequency (counting "um", "uh", "like", "you know", and similar tokens), average words per answer, and speaking pace in words per minute. These are measured directly from the Web Speech API transcript and per-answer duration, not estimated by an LLM.
- **Visual metrics.** Eye contact (iris position relative to eye corners, combined with head pitch from nose-to-eye geometry), posture (shoulder symmetry and nose centring), engagement (composite of eye contact and head stability), and stress (composite of head movement variance, blink-rate anomaly, and eye avoidance). All four metrics are computed from MediaPipe face and pose landmarks, smoothed across a twelve-frame rolling window of approximately four hundred milliseconds.

The browser maintains per-session averages of all six metrics and submits them with the feedback request.

4.3. Final Report Synthesis (Cold Path)

The feedback synthesiser is a single LLM call that receives the full session transcript, the per-answer scores, and the multimodal averages, and returns a structured JSON object containing the final five-dimension report. The weighting is:

Dimension	Weight	Primary signals
Technical correctness	30 %	Per-answer correctness and depth
Communication clarity	20 %	Filler-word frequency, words per answer, pace
Vocal confidence	15 %	Hedging language, "I don't know" frequency, fillers
Engagement & eye contact	15 %	Eye-contact average, posture, head stability
Answer structure	20 %	STAR analysis on behavioural answers

The synthesiser is instructed to quote the candidate's literal words as evidence in each improvement bullet, tag improvements by severity (low / medium / high), produce a STAR breakdown for each behavioural answer with per-component sub-scores, and generate exactly three coaching steps, each under two hours of effort.

5. Key Design Choices and Trade-offs

Every architectural decision involved a deliberate trade-off. The most significant ones are tabulated in Table 1.

6. Limitations, Assumptions, and Next Steps

6.1. Assumptions

The system was designed under the following assumptions, all of which are acceptable for the hackathon context but would need to be relaxed for a production deployment:

- A single user runs a single session at a time; there is no concurrent-interview support.
- Prompts and speech recognition are English-only. The Web Speech API supports more than one hundred languages, but the LLM prompts do not.
- Résumé PDFs are text-extractable. Scanned or image-only PDFs would require OCR preprocessing that is not currently implemented.
- The candidate uses Chrome with functioning webcam and microphone permissions granted.

6.2. Known Limitations

- The computer-vision scoring is heuristic rather than learned. Accuracy degrades in poor lighting, with glasses, or with unusual camera angles.
- The LLM-derived "confidence" dimension overlaps partially with the measured filler-word

Table 1: Key design trade-offs.

Decision	Chosen approach	Trade-off accepted
Computer vision	Browser MediaPipe with heuristic scoring	Free and scalable, but less accurate than a learned model.
Speech to text	Web Speech API	Real-time and free, but lower accuracy on accented speech.
Session storage	In-memory for live sessions; SQLite for completed sessions	Zero infrastructure; single-host only.
Feedback synthesis	Single LLM call in JSON mode	Faster and atomic, but a single point of failure.
Orchestrator	LangGraph state machine	Explicit nodes and easy to extend, but adds a dependency.
Job feed	Adzuna public API	Terms-of-service compliant; limited to Adzuna's crawled feeds.
Embeddings	sentence-transformers (local)	Zero API cost; lower ceiling than hosted models.
LLM provider	Gemini 2.5 Flash (Lite for hot path, Pro for synthesis)	Generous free tier (1000 requests per day on Flash Lite) sufficient for demo workload.

count. Ideally the two sources would be cleanly separated.

- There is no real prosody model. Tone and pitch analysis are approximated by the LLM scoring the transcript text rather than by an actual acoustic-feature model.
- Sessions are persisted to SQLite for history, but the live session state is in-memory and is lost on backend restart mid-interview.

6.3. Next Steps

A six-month roadmap to take this from hackathon prototype to production would address the limitations above and add several features that were out of scope:

- **Persistence layer.** Migrate session storage to Postgres with Redis for transient state. Add user accounts.
- **Trained computer-vision model.** Fine-tune a small classifier head on labelled interview footage to replace the heuristic eye-contact and posture scoring.
- **Whisper-based speech-to-text fallback.** Add a server-side Whisper deployment as an opt-in path for users with accents that Web Speech misses.
- **Acoustic prosody model.** Extract pitch variance, speech rate, and vocal energy with openSMILE or a small CNN. Feed these into the confidence dimension as a measured signal rather than an LLM estimate.
- **Recruiter mode.** Allow hiring managers to define custom assessments and have candidates take them asynchronously, with a structured rubric output. This is the natural revenue model for the platform.
- **Multi-language support.** Extend prompts and TTS voice selection to Hindi, Tamil, Spanish, and Mandarin via prompt templates.
- **Coding round.** Add a Pyodide-based browser Python execution sandbox with auto-graded test cases and LLM code review for technical roles.